

OSCAR INSIGHT SEARCH

Sistema de Recuperación de Información Cinematográfica

Informe Técnico

Curso	SRI 2025-2026
Corte actual	Corte 1 (Adquisición e Indexación) + Corte 2 (Recuperación)
Dominio	Cine — Películas en inglés (1990–2026), Premios Oscar
Modelo base	Booleano Extendido (Extended Boolean Model, EBM)
Modelo híbrido	EBM (p-norm tf-idf) + Semántico vectorial (FAISS + MiniLM)
Corpus generado	1,623+ documentos 16,381 términos únicos en vocabulario
Stack tecnológico	Python 3.11 FastAPI NLTK FAISS sentence-transformers
Referencias	Baeza-Yates & Ribeiro-Neto (2011); Salton et al. (1983)

1. Abstract

Oscar Insight Search es un **Sistema de Recuperación de Información (SRI)** de dominio restringido orientado al cine y los Premios Oscar. El sistema integra datos de **TMDB API v3** (metadatos estructurados: director, reparto, géneros, presupuesto, recaudación) y **Metacritic** (reseñas de crítica en lenguaje natural) para construir un corpus cinematográfico de 1,623+ documentos con 16,381 términos únicos en el vocabulario del índice invertido.

El sistema implementa dos modelos de recuperación complementarios: (1) el **Modelo Booleano Extendido (EBM)** basado en la p-norma euclídea con ponderación tf-idf normalizado (Salton et al., 1983; Baeza-Yates & Ribeiro-Neto, 2011, Cap. 3), y (2) un **motor semántico vectorial** basado en embeddings densos generados con el modelo `sentence-transformers/all-MiniLM-L6-v2` e indexados con **FAISS** (IndexFlatIP con similitud coseno). Ambos motores se combinan en un **motor híbrido** con ponderación configurable (60% EBM / 40% vectorial) servido a través de una API REST construida con FastAPI.

2. Dominio del Conocimiento

2.1 Caracterización del Corpus

El dominio está acotado al cine comercial en lengua inglesa producido entre 1990 y 2026, con énfasis en películas de alta visibilidad (candidatas o ganadoras de los Premios Oscar u otras premiaciones relevantes). La elección de este dominio responde a tres criterios de diseño:

- Alta disponibilidad de datos estructurados (TMDB API) y texto en lenguaje natural (reseñas de crítica).
- Vocabulario técnico-artístico estable y definido (géneros, directores, actores, premios).
- Existencia de dos tipos de documentación complementaria: metadatos cuantitativos (presupuesto, recaudación, calificación) y texto cualitativo (sinopsis, reseñas).

Atributo	Descripción
Dominio	Cine (películas en inglés, 1990–2026)
Fuente primaria	TMDB API v3 (metadatos estructurados)
Fuente secundaria	Metacritic (reseñas de crítica en lenguaje natural)
Documentos indexados	1,623+ películas
Términos en índice	16,381 términos únicos (stems Snowball)
Idioma	Inglés (stop-words + stemming Snowball)
Cobertura temporal	1990 – 2026

2.2 Esquema de Documento (Schema v2)

Cada documento del corpus tiene la siguiente estructura JSON, denominada internamente schema v2 (compatible hacia atrás con el schema v1 de Letterboxd/Wikipedia):

Estructura del Documento (schema v2)

```
{ "title": "Oppenheimer", "year": "2023",  
  "metadata": { "director": "Christopher Nolan",  
    "cast": ["Cillian Murphy", "Emily Blunt", "Matt Damon"],  
    "genres": ["Drama", "History"], "budget": 100000000,  
    "revenue": 952000000, "vote_average": 8.1, "vote_count": 14200,  
    "imdb_id": "tt15398776", "tmdb_id": 872585,  
    "source_url": "https://www.themoviedb.org/movie/872585",  
    "tagline": "The world forever changes." },  
  "rich_text": "Oppenheimer. Drama History. Christopher Nolan. [sinopsis]. [reseñas...]",  
  "reviews_count": 10 }
```

El campo **rich_text** es el campo principal para la indexación. Concentra todos los campos textuales del documento en un único string que es procesado por el pipeline de normalización lingüística. Su construcción sigue un orden de importancia descendente: título (con doble peso para incrementar su tf) → tagline → géneros → director → reparto → sinopsis completa → reseñas de crítica.

3. Arquitectura General del Sistema

3.1 Visión de Módulos

El sistema está organizado en cinco módulos con responsabilidades bien delimitadas:

Módulo	Componente clave	Responsabilidad
crawler/	tmdb_client.py, scraper.py	Adquisición de datos desde TMDB API v3 y Metacritic (web scraping con TLS fingerprinting)
indexer/	inverted_index.py, ebm.py	Pipeline NLP, construcción del índice invertido y cálculo de pesos tf-idf para EBM
database/	store.py, checkpoint.py, vector_store.py	Persistencia JSON del corpus, checkpointing incremental e índice vectorial FAISS
api/	main.py	Motor de recuperación híbrido EBM+Vectorial servido como API REST (FastAPI)
scripts/	populate_tmdb.py, query.py	Orquestación de la población del corpus y CLI de consulta directa al índice

3.2 Pipeline de Procesamiento End-to-End

El flujo completo de datos desde la adquisición hasta la recuperación es el siguiente:

- TMDB API /discover/movie → Lista básica de películas (tmdb_id, título, año, overview, vote_count) con filtros de calidad (vote_count ≥ 100, idioma inglés).
- TMDB API /movie/{id}?append_to_response=credits,external_ids → Enriquecimiento: director, cast (top 10), géneros, presupuesto, recaudación, imdb_id, tagline.
- Metacritic /movie/{slug}/user-reviews/ → Scraping con TLS fingerprinting (curl_cffi, impersonate=chrome124) para obtener hasta 10 reseñas largas (>100 caracteres) por película.
- build_rich_text() → Concatenación ordenada de todos los campos textuales en el campo rich_text del documento.
- InvertedIndex.add_film() → Pipeline NLP: lower → word_tokenize → filtro alfanumérico → stop-words (EN+ES) → SnowballStemmer → Counter(tf).
- Checkpointing → Guardado incremental cada 25 documentos (checkpoint.json) y volcado a disco cada 50 documentos (documents.json + index.json).
- ExtendedBooleanModel.build_weights() → Cálculo de pesos tf-idf normalizados $w_{\{i,j\}}$ para todos los términos e indexación FAISS con all-MiniLM-L6-v2.
- FastAPI /search → Motor híbrido: EBM (p-norm OR/AND) + FAISS (similitud coseno) combinados en score ponderado, snippet extraction y respuesta JSON.

4. Módulo de Adquisición (Crawler)

4.1 Cliente TMDb API v3 (tmdb_client.py)

El cliente `TmdbClient` implementa un wrapper sobre la TMDb REST API v3 con gestión de autenticación dual (`api_key` como query param o `Authorization: Bearer` header), rate limiting conservador y sesión HTTP persistente (`requests.Session`).

4.1.1 Estrategias de Descubrimiento

El sistema implementa dos estrategias complementarias de descubrimiento de películas, seleccionables en tiempo de ejecución:

Estrategia	Parámetro TMDb	Filtro adicional	Objetivo
A — Popularity	<code>sort_by=popularity.desc</code>	<code>vote_count.gte=100</code>	Películas muy conocidas con alta probabilidad de tener reseñas en Metacritic
B — Quality	<code>sort_by=vote_average.desc</code>	<code>vote_count.gte=500</code>	Películas de alta calidad crítica, independientemente de su popularidad masiva

Ambas estrategias aplican un filtro adicional de cobertura temporal (`primary_release_date.gte=1990-01-01`) y restricción de idioma original (`with_original_language=en`) para maximizar la cobertura de reseñas en Metacritic.

4.1.2 Rate Limiting

El tier gratuito de TMDb permite aproximadamente 40 peticiones por cada 10 segundos. El cliente implementa un rate limiter conservador mediante `time.sleep()` con un **delay de 0.27 segundos entre peticiones** (≈ 37 req/10s), dejando un margen de seguridad del 7.5% respecto al límite máximo. El uso de `requests.Session` permite la reutilización de conexiones TCP (keep-alive), reduciendo la latencia acumulada en corpus grandes.

4.1.3 Enriquecimiento en Una Sola Petición

El método `get_movie_details(tmdb_id)` usa el parámetro `append_to_response=credits,external_ids` de TMDb para obtener los créditos completos (director, cast) y los IDs externos (`imdb_id`) en una única llamada HTTP, reduciendo el número de peticiones en un factor de 3x respecto a llamadas separadas.

4.2 Scraper de Reseñas — Metacritic (scraper.py)

El componente `MetacriticReviewScraper` extrae reseñas de usuarios de Metacritic superando la protección WAF/Cloudflare mediante **TLS fingerprinting** a través de la biblioteca `curl_cffi` con `impersonate="chrome124"`. Este enfoque replica el handshake TLS exacto de Chrome 124, haciendo indistinguible el cliente automatizado de un navegador real a nivel de capa de transporte.

4.2.1 Técnica de Evasión: TLS Fingerprinting

La protección Cloudflare y otros WAF modernos identifican clientes automatizados no solo por los User-Agent headers HTTP, sino también por las características del handshake TLS: cipher suites ofertadas, extensiones TLS, orden de parámetros, etc. La librería `curl_cffi` expone la API de libcurl con soporte para impersonation de browsers reales, permitiendo replicar exactamente el fingerprint JA3/JA4 del browser objetivo.

- `curl_cffi.requests.Session(impersonate="chrome124")` replica el TLS fingerprint de Chrome 124.
- Headers HTTP completos de Chrome real (User-Agent, Accept, Accept-Language, Upgrade-Insecure-Requests).
- Warmup inicial al cargar la sesión para establecer cookies y estado de sesión.
- Delays aleatorios entre peticiones (1.5–3.5 segundos) para imitar comportamiento humano.
- Sistema de reintentos con backoff exponencial (hasta 2 reintentos con delays incrementales).

4.2.2 Resolución Heurística de Slugs

Metacritic usa slugs de URL en formato kebab-case para identificar películas (ej. `/movie/oppenheimer/user-reviews/`). El sistema implementa la función `_slugify()` que normaliza el título mediante: (1) normalización Unicode NFKD para separar diacríticos, (2) conversión a ASCII eliminando caracteres no representables, (3) conversión a minúsculas, (4) eliminación de caracteres no alfanuméricos, y (5) sustitución de espacios por guiones. Adicionalmente, se prueban variaciones con el año añadido al slug (`oppenheimer-2023`) para manejar películas con títulos duplicados.

4.2.3 Extracción de Reseñas y Filtros de Calidad

El parser HTML (BeautifulSoup + lxml) extrae reseñas aplicando un filtro de calidad estricto: solo se retienen textos de entre **100 y 3,000 caracteres**, excluyendo elementos de UI de Metacritic (textos con "Expand", referencias al sitio, prompts de login). Se implementa un doble mecanismo de extracción: primero se buscan en elementos `<span>` (estrategia principal), y como fallback se buscan en divs con clases específicas de reseña de Metacritic.

5. Módulo de Indexación

5.1 Índice Invertido (inverted_index.py)

La clase `InvertedIndex` implementa la estructura de datos central del SRI: el **índice invertido**, que mapea cada término normalizado del vocabulario a una lista de postings (`dict[str, dict[int, int]]`), donde cada posting es un par (`doc_id`, `tf`). Internamente se usa un `defaultdict(dict)` para eficiencia en la construcción incremental, expuesto públicamente como `dict[str, list[tuple[int, int]]]` ordenado por `doc_id`.

5.2 Pipeline de Normalización Lingüística

El método `_tokenize()` implementa un pipeline de cinco etapas secuenciales sobre el texto crudo:

#	Etapas	Implementación	Justificación
1	Lowercasing	<code>text.lower()</code>	Normalización léxica: 'Oscar' == 'oscar'
2	Tokenización	<code>nltk.word_tokenize()</code>	Tokenización lingüística (maneja contracciones, puntuación)
3	Filtro alfanumérico	<code>re.search(r'[a-z0-9]', token)</code>	Elimina puntuación pura y caracteres especiales
4	Stop-words	<code>frozenset (EN + ES)</code>	Elimina términos sin valor discriminativo (artículos, preposiciones)
5	Stemming	<code>SnowballStemmer('english')</code>	Reduce variantes morfológicas al stem raíz (<code>director/directing</code> → <code>direct</code>)

Las **stop-words bilingües** (inglés + español) cargadas desde NLTK corpus cubren aproximadamente 350 términos y se almacenan como `frozenset` para garantizar tiempo de lookup $O(1)$. El **SnowballStemmer** (también conocido como Porter2) es una evolución del algoritmo de Porter que produce stems más agresivos y consistentes, reduciendo el vocabulario efectivo en un factor típico de 3–4x. Por ejemplo, "cinematography", "cinematographic" y "cinematographer" convergen al stem "cinematograph".

5.3 Estructura de Datos: Posting Lists

Internamente, el índice usa la estructura:

Estructura interna del índice invertido

```
self._raw_index: defaultdict[str, dict[int, int]]
{ 'oppenheim': {0: 3, 2: 1, 14: 2}, ... }
```

```
{ 'cinematograph': {5: 1, 89: 4}, ... }
```

Propiedad pública `.index`: `dict[str, list[tuple[int, int]]]`

```
{ 'oppenheim': [(0, 3), (2, 1), (14, 2)], ... }
```

Esta representación permite calcular directamente la **frecuencia del término (tf)** por documento, necesaria para el cálculo posterior de los pesos tf-idf del modelo EBM. La elección de `dict[int, int]` como posting list interna (en lugar de `list[tuple[int, int]]`) garantiza tiempo $O(1)$ para actualizar la frecuencia de un término en un documento ya indexado, lo cual es crítico para el proceso de población incremental con checkpointing.

5.4 Método `add_film()` — Indexación Compatible con Ambos Esquemas

El método `add_film(doc_id, film_data)` actúa como adaptador entre el esquema de documento y el índice. Para **schema v2** (TMDb), usa directamente el campo precomputado `rich_text`. Para **schema v1** (legacy Wikipedia/Letterboxd), construye el texto indexable concatenando los campos disponibles (title, synopsis, director, genre, cast, reviews). Esta dualidad garantiza compatibilidad retroactiva con corpora anteriores sin necesidad de migración.

6. Modelo Booleano Extendido (EBM)

6.1 Fundamento Teórico

El **Modelo Booleano Extendido** (Salton, Fox & Wu, 1983; formalizado en Baeza-Yates & Ribeiro-Neto, 2011, Cap. 3) generaliza el modelo booleano clásico superando su principal limitación: la incapacidad de producir un ranking de documentos relevantes. El modelo booleano clásico solo puede determinar si un documento satisface o no una consulta (relevancia binaria), mientras que el EBM asigna un **score continuo** $\in [0, 1]$ a cada documento mediante distancias euclidianas ponderadas por tf-idf.

6.2 Ponderación TF-IDF Normalizada

La clase `ExtendedBooleanModel.build_weights()` calcula el peso $w_{\{i,j\}}$ para cada par (término i , documento j):

Fórmula de ponderación TF-IDF normalizada

$$w_{\{i,j\}} = \text{norm_tf}_{\{i,j\}} * \text{idf}_i$$

Donde:

$$\text{norm_tf}_{\{i,j\}} = \text{tf}_{\{i,j\}} / \text{max_tf}_j$$

$\text{tf}_{\{i,j\}}$ = frecuencia del término i en el documento j

max_tf_j = max. frecuencia de cualquier término en el documento j

$$\text{idf}_i = \log(N / n_i) / \log(N) \quad [\text{normalizado al rango } [0,1]]$$

N = número total de documentos en el corpus

n_i = número de documentos que contienen el término i

Caso especial: si $n_i \geq N \rightarrow \text{idf}_i = 0.01$ (términos omnipresentes)

La normalización del tf por el **máximo tf del documento** (norma de Ltn) evita que documentos más largos reciban un sesgo artificial. La normalización del idf por $\log(N)$ (en lugar del idf estándar) restringe el peso al rango $[0, 1]$, haciendo los scores comparables entre diferentes consultas y facilitando la combinación con los scores del modelo vectorial.

6.3 Función de Similitud P-Norm

La función de similitud del EBM generaliza los operadores AND/OR booleanos mediante la p-norma euclídea ($p=2$ por defecto, configuración estándar de Salton et al.):

Funciones de similitud EBM (p-norm)

Para una consulta q con m términos $\{t_1, t_2, \dots, t_m\}$:

Consulta OR (disyunción):

$$\text{sim_OR}(d, q) = \left(\text{SUM}(w_i^p) / m \right)^{1/p}$$

Consulta AND (conjunción):

$$\text{sim_AND}(d, q) = 1 - \left(\text{SUM}((1 - w_i)^p) / m \right)^{1/p}$$

Donde $w_i = w_{\{i,j\}}$ es el peso tf-idf normalizado del término t_i en el documento d_j y p es el exponente de la norma (default $p=2.0$, configurable en la API).

La interpretación geométrica es directa: **sim_OR** mide qué tan cerca está el vector de pesos del documento del vector ideal $(1, 1, \dots, 1)$ — máxima relevancia. **sim_AND** mide qué tan lejos está del vector nulo $(0, 0, \dots, 0)$. Para $p=2$, ambas funciones corresponden a distancias euclidianas normalizadas, lo que las hace equivalentes a similitudes coseno bajo ciertas condiciones.

6.4 Implementación del Motor de Búsqueda EBM

El método `ExtendedBooleanModel.search(query, op)` implementa el motor de recuperación en cuatro etapas: (1) tokenización de la consulta con el mismo pipeline NLP del índice (garantizando consistencia de stems), (2) identificación de documentos candidatos (unión de las posting lists de todos los términos de la consulta), (3) cálculo de la similitud p-norm para cada documento candidato usando los pesos precomputados, y (4) ordenamiento descendente por score y retorno de resultados. Los pesos se persisten en `data/ebm_weights.json` para evitar recalcularlos en cada arranque del servidor.

7. Módulo de Búsqueda Semántica Vectorial

7.1 Arquitectura del Vector Store

El módulo `database/vector_store.py` implementa la capa de recuperación semántica basada en **embeddings densos**. A diferencia del EBM (recuperación léxica basada en frecuencias de términos), los modelos de embeddings capturan la **semántica distribucional** del texto, permitiendo encontrar documentos relevantes incluso cuando no comparten vocabulario exacto con la consulta.

7.2 Modelo de Embeddings: all-MiniLM-L6-v2

El sistema utiliza el modelo `sentence-transformers/all-MiniLM-L6-v2`, un transformer de 6 capas y 22 millones de parámetros que produce embeddings de **384 dimensiones**. Este modelo fue fine-tuneado sobre 1,000M+ pares de oraciones con el objetivo de aprendizaje contrastivo, haciendo que oraciones semánticamente similares tengan vectores cercanos en el espacio latente. Es el modelo estándar de la familia sentence-transformers por su balance entre calidad (correlation de Spearman ~63% en benchmarks STS) y velocidad de inferencia.

- Cada documento se vectoriza truncando el `rich_text` a los primeros 2,000 caracteres para no saturar el contexto del transformer (512 tokens máx.).
- Los vectores se normalizan L2 antes de la indexación en FAISS para que el producto interno sea equivalente a la similitud coseno.
- El encoding se realiza en batch (`show_progress_bar=True`) con numpy para maximizar el throughput en GPU o CPU.

7.3 Índice FAISS (IndexFlatIP)

Los vectores normalizados se indexan con **FAISS IndexFlatIP** (Facebook AI Similarity Search, Inner Product). La elección de IndexFlatIP sobre IndexFlatL2 es deliberada: dado que los vectores están normalizados L2, el **producto interno es equivalente a la similitud coseno**, lo que simplifica la interpretación de los scores. IndexFlatIP realiza búsqueda exhaustiva exacta (no aproximada), garantizando precisión máxima al costo de tiempo $O(n \cdot d)$ donde n es el número de documentos y d la dimensión del embedding.

El sistema mantiene dos diccionarios de mapeo bidireccional: `vector_to_doc: dict[int, int]` (índice FAISS → `doc_id` del corpus) y `doc_to_vector: dict[int, int]` (`doc_id` → índice FAISS). Esta dualidad permite tanto búsquedas (`query` → k documentos más similares) como recuperaciones directas por `doc_id`. El índice se persiste en formato binario FAISS (`data/faiss_index.bin`, ≈2.5 MB para 1,623 documentos) con los mappings en JSON.

8. Motor de Recuperación Híbrido (API REST)

8.1 Diseño de la API FastAPI

La API REST está implementada con **FastAPI** y servida con Uvicorn (ASGI). FastAPI proporciona validación automática de esquemas de entrada/salida mediante **Pydantic v2**, documentación interactiva OpenAPI/Swagger autogenerada en `/docs`, y soporte nativo para programación asíncrona (`async def`).

Endpoint	Método	Descripción
GET /	GET	Health check: retorna status, modelo activo, número de documentos indexados y tamaño del vocabulario
POST /search	POST	Motor de recuperación híbrido EBM+Vectorial. Acepta query, top_k (1-50), exponente p (p-norm) y flag hybrid

8.2 Algoritmo de Recuperación Híbrida

El endpoint POST /search implementa el siguiente algoritmo de fusión de scores:

9. Búsqueda EBM: `ExtendedBooleanModel.search(query, op='OR')` → lista de (doc_id, ebm_score) usando la p-norm sobre pesos tf-idf precomputados.
10. Búsqueda vectorial: `VectorStore.search(query, top_k=50)` → embeddings de la consulta con all-MiniLM-L6-v2, búsqueda FAISS top-50, retorna (doc_id, cosine_score).
11. Fusión de scores: Para cada doc_id en la unión de ambos conjuntos de resultados:
 $h_score = (s_ebm \times 0.6) + (s_vec \times 0.4)$.
12. Snippet extraction: Para cada resultado, se busca la primera ocurrencia en el rich_text de algún stem de la consulta y se extrae un fragmento de ± 200 caracteres alrededor.
13. Ordenamiento final: Descendente por h_score, se retornan los top_k resultados con metadatos completos.

8.3 Justificación de la Ponderación 60/40

La ponderación **60% EBM / 40% vectorial** responde a las características del dominio: el corpus cinematográfico tiene un vocabulario técnico bien definido (nombres de directores, géneros, títulos) donde la coincidencia léxica exacta (EBM) es altamente informativa. El componente vectorial aporta la capacidad de recuperar documentos semánticamente relacionados aun cuando no comparten términos exactos con la consulta (sinonimia, paráfrasis). Esta combinación híbrida es un patrón establecido en SRI moderno (cf. **BM25 + Dense Retrieval** en sistemas como DPR y ColBERT).

9. Módulo de Almacenamiento y Persistencia

9.1 DocumentStore (store.py)

La clase `DocumentStore` gestiona la persistencia del corpus en formato JSON plano, sin dependencia de bases de datos externas. Esta decisión de diseño reduce las dependencias del sistema y simplifica el despliegue, a costa de tiempos de carga proporcionales al tamaño del corpus ($\approx 2s$ para 1,623 documentos en HDD estándar).

9.1.1 Deduplicación Multicapa

El store implementa un sistema de deduplicación en dos niveles para evitar indexar la misma película más de una vez:

- Nivel primario: Índice hash en memoria `tmdb_id` $\rightarrow$ `doc_id` ($O(1)$ lookup). Si el `tmdb_id` ya existe, se retorna el `doc_id` existente sin crear una nueva entrada.
- Nivel secundario: Índice hash `source_url` $\rightarrow$ `doc_id` como fallback para películas sin `tmdb_id` (schema v1 legacy).

9.1.2 Formato de Persistencia

El corpus se serializa en `data/documents.json` con un objeto `_meta` que almacena el total de documentos y el siguiente `doc_id` autoincremental. El índice invertido se serializa por separado en `data/index.json` como `dict[str, list[list[int, int]]]` (las tuplas se serializan como arrays JSON). El tamaño del corpus serializado es de ≈ 9.8 MB (`documents.json`) y ≈ 18 MB (`index.json`) para 1,623 documentos.

9.2 Sistema de Checkpointing (checkpoint.py)

La clase `Checkpoint` implementa un mecanismo de reanudación para procesos de población largos (2,000+ películas $\times$ $\sim 3s$ por película ≈ 100 minutos). El estado se persiste en `data/checkpoint.json` con los siguientes campos:

- `processed_tmdb_ids`: Set de IDs ya procesados exitosamente (guardado como lista ordenada para reproducibilidad).
- `failed_tmdb_ids`: IDs que fallaron en el scraping (para reintentos selectivos con `--retry-failed`).
- `last_page_popularity` / `last_page_quality`: Última página procesada de cada estrategia.
- `total_indexed`: Contador acumulativo de documentos indexados.

El checkpoint se guarda cada **CHECKPOINT_EVERY=25 documentos** y el corpus completo se vuelca a disco cada **SAVE_DB_EVERY=50 documentos**. Esta asimetría garantiza que una interrupción pierda como máximo 25 documentos de progreso pero mantiene el I/O de escritura a un nivel manejable.

10. Orquestación del Corpus (populate_tmdb.py)

10.1 Pipeline de Construcción del rich_text

La función `build_rich_text(details, reviews)` construye el campo indexable del documento concatenando los campos en un orden de prioridad descendente por peso informativo:

#	Campo	Justificación del peso
1	Título (×2)	Doble aparición para incrementar artificialmente el tf del título y maximizar su peso en el índice
2	Tagline	Alta densidad informativa en pocas palabras; a menudo captura el tono temático de la película
3	Géneros	Vocabulario controlado de alta discriminabilidad (Drama, Thriller, Comedy...)
4	Director	Named entity de alta especificidad para consultas orientadas a autor
5	Reparto (top 10)	Named entities de alta frecuencia en consultas del usuario
6	Sinopsis completa	Texto narrativo denso; fuente principal de términos temáticos
7	Reseñas (hasta 10)	Lenguaje natural variado y subjetivo; amplía la cobertura del vocabulario con sinónimos y paráfrasis

10.2 Estrategia de Población Doble (A+B)

El orquestador ejecuta primero la estrategia A (`popularity.desc`) para las primeras N páginas (N configurable, default 100) y luego la estrategia B (`vote_average.desc`) para las páginas restantes. La deduplicación del DocumentStore garantiza que las películas que aparecen en ambas estrategias no se indexan dos veces.

10.3 CLI y Modos de Ejecución

Argumento CLI	Comportamiento
<code>--pages N</code>	Número de páginas TMDb a procesar (20 films/página × N páginas ≈ 20N documentos)
<code>--resume</code>	Reanuda desde el estado guardado en <code>checkpoint.json</code> (omite IDs ya procesados)
<code>--reset</code>	Borra <code>checkpoint</code> y <code>corpus</code> ; reinicia desde cero
<code>--no-reviews</code>	Omite el scraping de Metacritic (modo rápido para pruebas, ~2 min/100 films)
<code>--api-key / TMDB_API_KEY</code>	API key de TMDb (arg CLI o variable de entorno)

11. Despliegue y Containerización

11.1 Dockerfile Multietapa

El sistema incluye un **Dockerfile multietapa** (builder + runtime) optimizado para minimizar el tamaño de la imagen final. La etapa `builder` instala todas las dependencias de compilación (build-essential, gcc para lxml y curl_cffi) en una imagen temporal. La etapa `runtime` copia solo los binarios instalados (sin las herramientas de compilación), reduciendo el tamaño final en ≈60%.

- Python 3.11-slim como imagen base (≈130 MB vs ≈900 MB de la imagen completa).
- Datos NLTK pre-descargados en tiempo de build (punkt, punkt_tab, stopwords) y copiados a /usr/share/nltk_data para evitar descargas en runtime.
- Variable NLTK_DATA configurada como variable de entorno para que NLTK la detecte automáticamente.
- El directorio data/ se monta como volumen externo (no incluido en la imagen) para separar código y datos.
- Puerto 8000 expuesto, con Uvicorn como servidor ASGI en modo producción (--host 0.0.0.0).

11.2 Estrategia de Datos como Volumen

Los archivos de datos generados (`data/documents.json`, `data/index.json`, `data/faiss_index.bin`, `data/ebm_weights.json`) se montan como volumen Docker externo: `docker run -v ${PWD}/data:/app/data oscar-insight:0.2`. Esta separación permite actualizar el código sin invalidar el corpus generado, y viceversa.

11.3 Stack de Dependencias

Biblioteca	Versión mínima	Función
beautifulsoup4	≥4.12.3	Parsing HTML de páginas de Metacritic
requests	≥2.31.0	Cliente HTTP para TMDb API
lxml	≥5.3.0	Parser HTML rápido para BeautifulSoup
curl_cffi	≥0.7.0	TLS fingerprinting (evasión Cloudflare)
nltk	≥3.8.1	Tokenización, stop-words, SnowballStemmer
scikit-learn	≥1.3.0	Utilidades NLP y métricas de evaluación
sentence-transformers	≥3.3.0	Embeddings densos (all-MiniLM-L6-v2)
faiss-cpu	≥1.9.0	Índice vectorial (IndexFlatIP)
numpy	≥1.26.0	Operaciones matriciales para embeddings
fastapi	≥0.115.0	Framework API REST asíncrono
uvicorn[standard]	≥0.30.0	Servidor ASGI para FastAPI

Biblioteca	Versión mínima	Función
pydantic	≥2.10.0	Validación de esquemas de entrada/salida

12. Estado del Proyecto y Roadmap

12.1 Componentes Implementados

Corte	Módulo	Descripción	Estado
1	crawler/tmdb_client.py	Cliente TMDB API v3 con estrategias A/B y rate limiting	Implementado
1	crawler/scrapper.py	Scraper Metacritic con TLS fingerprinting (curl_cffi)	Implementado
1	indexer/inverted_index.py	Índice invertido + pipeline NLTK (5 etapas)	Implementado
1	database/store.py	DocumentStore JSON con deduplicación multicapa	Implementado
1	database/checkpoint.py	Checkpointing incremental con reanudación	Implementado
1	Corpus generado	1,623 películas, 16,381 términos únicos	Generado
2	indexer/ebm.py	Motor EBM: pesos tf-idf normalizados + p-norm	Implementado
2	database/vector_store.py	FAISS IndexFlatIP + MiniLM embeddings	Implementado
2	api/main.py	API REST híbrida EBM+Vectorial (FastAPI)	Implementado
3	Interfaz web	Frontend React/Vue para consultas interactivas	Pendiente
3	RAG pipeline	Retrieval-Augmented Generation para respuestas enriquecidas	Pendiente

12.2 Componentes Pendientes (Corte 3)

- Interfaz web interactiva (React o Vue.js) para consultas en lenguaje natural con visualización de resultados, scores EBM/vectorial y snippets.
- Pipeline RAG (Retrieval-Augmented Generation): usar los top-k documentos recuperados como contexto para un LLM que genere respuestas enriquecidas en lenguaje natural ('¿Qué películas de Nolan ganaron el Oscar a mejor director?').
- Evaluación formal con métricas IR estándar: Precision@k, Recall@k, MAP (Mean Average Precision), NDCG.
- Expansión del corpus a 5,000+ documentos con más fuentes (IMDb reviews, Wikipedia, Rotten Tomatoes).

13. Análisis Crítico y Limitaciones

13.1 Fortalezas del Diseño

- Separación de responsabilidades clara entre módulos (adquisición / indexación / almacenamiento / recuperación / API).
- Pipeline NLP robusto con stop-words bilingües y stemming Snowball, adecuado para un corpus predominantemente en inglés con metadatos mixtos.
- Motor de recuperación híbrido que combina la precisión léxica del EBM con la generalización semántica del modelo vectorial.
- Sistema de checkpointing que permite interrumpir y reanudar procesos largos sin pérdida de progreso significativa.
- Containerización Docker con imagen multietapa optimizada y separación código/datos mediante volúmenes.

13.2 Limitaciones y Áreas de Mejora

- El DocumentStore JSON no escala eficientemente más allá de ~50,000 documentos; para corpora mayores se recomienda migrar a SQLite, PostgreSQL o Elasticsearch.
- La búsqueda FAISS con IndexFlatIP es exacta pero $O(n \cdot d)$, lo que puede ser lento para corpus >100,000 documentos. IndexIVFFlat o HNSW ofrecerían búsqueda aproximada con mejora de velocidad 10-100x.
- El scraper de Metacritic es inherentemente frágil ante cambios en la estructura HTML del sitio. Un cambio de selectores CSS o de la estructura del DOM puede romper la extracción sin aviso.
- La ponderación híbrida 60/40 es fija y no adaptativa. Un enfoque de aprendizaje (Learning to Rank) con datos de relevancia etiquetados produciría mejores resultados.
- La evaluación del sistema es actualmente manual (no hay gold standard de consultas y juicios de relevancia). La implementación de un test set evaluado con métricas IR estándar es imprescindible.
- El campo rich_text trunca las reseñas de Metacritic a los primeros 2,000 caracteres para el vectorizador, lo que puede perder información relevante en reseñas largas.

13.3 Comparación con Modelos Alternativos

Modelo	Ventaja vs. EBM	Desventaja vs. EBM
Booleano clásico	Simplicidad extrema, determinismo total	Sin ranking, sin tolerancia a la imprecisión del usuario
BM25 (Okapi)	Mejor manejo de longitud de documentos (parámetro b), más robusto empíricamente	No captura semántica, vocabulario gap problem
VSM (coseno)	Ranking continuo, bien estudiado matemáticamente	Alta dimensionalidad (sparse), mismos problemas léxicos que EBM

Modelo	Ventaja vs. EBM	Desventaja vs. EBM
Dense Retrieval	Captura semántica profunda, sinónimos, paráfrasis	Requiere fine-tuning en el dominio, más costoso en inferencia
EBM+Vectorial (impl.)	Combina lo mejor de ambos paradigmas; configurable en tiempo de ejecución	Mayor complejidad de mantenimiento, dos modelos que sincronizar

14. Referencias Bibliográficas

1. Baeza-Yates, R., & Ribeiro-Neto, B. (2011). *Modern Information Retrieval: The Concepts and Technology behind Search* (2nd ed.). Addison-Wesley. [Fundamento teórico del EBM, Cap. 3-4]
2. Salton, G., Fox, E. A., & Wu, H. (1983). Extended Boolean information retrieval. *Communications of the ACM*, 26(11), 1022–1036. doi:10.1145/182.358466
3. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. [Pipeline NLTK: tokenización, stop-words, stemming]
4. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP 2019*. [Fundamento de sentence-transformers / all-MiniLM-L6-v2]
5. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547. [FAISS]
6. The Movie Database. (2024). *TMDB API v3 Documentation*. <https://developer.themoviedb.org/>
7. Porter, M. F. (2001). Snowball: A language for stemming algorithms. <https://snowballstem.org/>

— Fin del Informe Técnico-Científico —
Oscar Insight Search | SRI 2025-2026